

Chapter 1: Introduction

Operating System (OS) = phần mềm quản lý tài nguyên máy tính và cung cấp môi trường chạy chương trình.
Hardware ← OS ← Applications ← Users
Hệ thống máy tính được chia thành 4 thành phần cơ bản: Phần cứng (CPU, bộ nhớ, thiết bị I/O), Hệ điều hành, Chương trình ứng dụng và Người dùng.
Interrupt là cơ chế thiết bị báo CPU có sự kiện xảy ra (Keyboard nhập → interrupt, Disk đọc xong → interrupt ...)
Storage Hierarchy: Disk → RAM → Cache → Register (Copy dữ liệu từ chậm → nhanh)
Các Chức năng Quản lý Cốt lõi: Process Management, (Memory Management, Storage & Filesystem, Hệ thống I/O

Đa chương trình (Multiprogramming) cho phép giữ nhiều công việc trong bộ nhớ cùng lúc để đảm bảo CPU luôn có việc để thực thi, tăng hiệu quả sử dụng.
Đa nhiệm (Multitasking / Time-sharing) là cơ chế chuyển đổi CPU giữa các tiến trình cực kỳ nhanh chóng, tạo ra thời gian phản hồi ngắn để người dùng có thể tương tác.

Hệ thống hoạt động ở hai chế độ (Dual-mode) gồm chế độ người dùng (user mode = 1) và chế độ hạt nhân (kernel mode = 0), user mode gọi tới kernel mode thông qua System Call
User Mode Process memory, Code & data của process, System calls
Kernel Mode: Toàn bộ memory, Hardware / I/O devices, Process & scheduler, File system, Privileged instructions

Chapter 2: Operating-System Structures

System Calls cung cấp giao diện để truy cập dịch vụ OS, thường thông qua API thay vì gọi trực tiếp.
API phổ biến gồm: Win32, POSIX, Java API; thư viện C chuẩn thường cung cấp giao diện system call trên UNIX/Linux.
Có 3 cách truyền tham số cho system call: qua registers, qua memory block/table, qua stack
System calls gồm 6 nhóm: Process Control, File Management, Device Management, Information Maintenance, Communication, Protection

Trình liên kết (Linker) có nhiệm vụ kết hợp các tệp mã đối tượng (object files) cùng với các thư viện thành một tệp thực thi nhị phân duy nhất.
Trình nạp (Loader) chịu trách nhiệm đưa tệp thực thi từ ổ đĩa vào bộ nhớ chính để CPU có thể bắt đầu chạy chương trình.

Monolithic: Toàn bộ chức năng trong một kernel duy nhất → hiệu năng cao nhưng khó sửa đổi.
Layered: Chia OS thành nhiều lớp, mỗi lớp dùng lớp dưới → dễ debug nhưng giảm hiệu năng.
Microkernel: Kernel tối thiểu, dịch vụ đưa ra user space, giao tiếp bằng message passing → an toàn, dễ mở rộng nhưng chậm hơn.
Modules: Kernel gồm các module có thể load/unload động → linh hoạt, dễ mở rộng.
Hybrid: Kết hợp nhiều mô hình để cân bằng hiệu năng, bảo mật và tính linh hoạt.
MS-DOS, UNIX → Monolithic, Linux → Monolithic + Modules

Chapter 3 – Processes

Tiến trình (Process) là chương trình đang thực thi; program là thực thể thụ động còn process là thực thể chủ động.
Thành phần process gồm: Text (mã lệnh), Program Counter, Stack (biến cục bộ, tham số), Data (biến global), Heap (cấp phát động).
Trạng thái process: New → tạo, Ready → chờ CPU, Running → đang chạy, Waiting → chờ sự kiện/I/O, Terminated → kết thúc
Mỗi process được quản lý bởi PCB (Process Control Block) chứa thông tin như: PID, state, CPU registers, memory info,...
Lập lịch (Scheduling) nhằm tối đa sử dụng CPU bằng cách chuyển đổi nhanh giữa các process. Ready Queue: process sẵn sàng chạy, Wait Queue: process đang chờ sự kiện/I/O
Context Switch: Quá trình CPU chuyển từ process này sang process khác bằng cách lưu trạng thái process cũ và nạp trạng thái process mới.

Fork

Tạo tiến trình: Parent tạo child tạo thành cây process. UNIX dùng fork() + exec(), Windows dùng CreateProcess().
Kết thúc tiến trình: Process tự kết thúc bằng exit(), parent có thể ép child kết thúc bằng abort().
fork(): tạo tiến trình con bằng cách sao chép tiến trình cha. Sau khi fork(), cha và con chạy độc lập. Dùng chung: code chương trình, Khác nhau: PID khác nhau, vùng nhớ logic riêng, thanh ghi riêng, biến thay đổi không ảnh hưởng nhau.

Inter-Process Communication - IPC

Shared Memory: Các tiến trình dùng chung vùng nhớ để trao đổi dữ liệu, cần đồng bộ hóa để tránh xung đột. Tốc độ nhanh nhưng quản lý phức tạp.
Truyền thông điệp (Message Passing): Các tiến trình giao tiếp thông qua hai lệnh cơ bản là send() và receive() mà không cần dùng chung biến bộ nhớ. Việc giao tiếp có thể là đồng bộ (blocking) hoặc bất đồng bộ (non-blocking).
Blocking (đồng bộ): Tiến trình phải chờ thao tác hoàn tất mới tiếp tục.
Non-blocking (bất đồng bộ): Tiến trình không cần chờ, tiếp tục thực thi ngay.

Pipes hoạt động như một ống dẫn cho phép hai tiến trình giao tiếp với nhau.
Giao tiếp Client-Server: Sockets: Điểm cuối giao tiếp mạng, xác định bằng địa chỉ IP + port, cho phép các tiến trình trao đổi dữ liệu qua mạng. RPC: Cho phép gọi hàm/thủ tục trên máy khác như gọi hàm cục bộ, dùng stub để đóng gói và giải mã tham số.

Chapter 4: Threads & Concurrency

Hầu hết các ứng dụng hiện đại đều sử dụng đa luồng (multithreaded) vì việc tạo luồng tốn ít tài nguyên (light-weight) hơn rất nhiều so với việc tạo các tiến trình (heavy-weight process).
Cấu trúc: Trong một tiến trình đa luồng, các luồng dùng chung mã (code), dữ liệu (data) và tệp (files); nhưng mỗi luồng sẽ duy trì các thanh ghi (registers), con trỏ lệnh (PC) và ngăn xếp (stack) độc lập
4 Lợi ích chính của Đa luồng: Responsiveness, Resource Sharing, Economy, Scalability

Multicore Programming

Concurrency: nhiều tác vụ cùng tiến triển (không nhất thiết cùng lúc)
Parallelism: nhiều tác vụ chạy đồng thời trên nhiều core
Data Parallelism: chia dữ liệu, cùng phép toán
Task Parallelism: chia công việc khác nhau cho các thread
Định luật Amdahl (Amdahl’s Law): $speedup \leq \frac{1}{s + \frac{1-s}{N}}$ (Trong đó S là tỷ lệ mã chạy tuần tự, N là số lõi xử lý).

Multithreading

User Thread: quản lý ở user space, nhanh hơn
Kernel Thread: kernel quản lý, hỗ trợ chạy song song thực sự
Many-to-One: nhiều user thread → 1 kernel thread (1 block → tất cả block)
One-to-One: 1 user thread → 1 kernel thread (Linux, Windows)
Many-to-Many: nhiều user thread - nhiều kernel thread

Thread Library: API tạo/quản lý thread: Pthreads, Windows Threads, Java Threads

Chapter 5: CPU Scheduling Chu kỳ CPU-I/O: Quá trình thực thi của một tiến trình bao gồm các chu kỳ thực thi CPU (CPU burst) và chờ I/O (I/O burst) diễn ra luân phiên nhau.
Bộ lập lịch CPU (CPU Scheduler): Làm nhiệm vụ chọn một tiến trình từ hàng đợi sẵn sàng (ready queue) và cấp phát lõi CPU cho tiến trình đó.
Dispatcher: chuyển CPU cho tiến trình được chọn (context switch, chuyển user mode...).

Tiêu chí đánh giá lập lịch CPU Utilization ↑ : tận dụng CPU tối đa
Throughput ↑ : số tiến trình hoàn thành / đơn vị thời gian
Turnaround Time ↓ : thời gian hoàn thành tiến trình

Waiting Time ↓ : thời gian chờ trong ready queue
Response Time ↓ : thời gian phản hồi đầu tiên

Các thuật toán lập lịch CPU
FCFS (First Come First Served): Đến trước chạy trước (FIFO), đơn giản nhưng dễ gây Convoy Effect. Thường dùng trong hệ thống đơn giản, batch processing hoặc hàng đợi in ấn.
SJF (Shortest Job First): Chọn tiến trình có CPU burst nhỏ nhất, tối ưu waiting time trung bình nhưng khó dự đoán CPU burst. Phù hợp hệ thống batch nơi có thể ước lượng thời gian chạy.
SRTF (Shortest Remaining Time First): Phiên bản preemptive của SJF, tiến trình mới có thời gian còn lại nhỏ hơn sẽ giành CPU. Dùng trong hệ thống cần giảm waiting time và phản hồi nhanh cho job ngắn.
RR (Round Robin): Mỗi tiến trình chạy tối đa một time quantum q , hết q chuyển cuối hàng đợi, response time tốt cho hệ tương tác. Thường dùng trong hệ điều hành chia sẻ thời gian (time-sharing), desktop OS.
Priority Scheduling: Chạy tiến trình ưu tiên cao hơn, có thể gây starvation, giải quyết bằng aging. Dùng cho hệ thống thời gian thực hoặc ưu tiên tác vụ quan trọng.
Multilevel Queue (MLQ): Chia nhiều queue theo loại tiến trình, queue ưu tiên cao chạy trước. Thường dùng để tách foreground/background hoặc interactive/batch processes.
Multilevel Feedback Queue (MLFQ): Tiến trình có thể di chuyển giữa các queue, hỗ trợ aging và linh hoạt hơn MLQ. Được dùng trong nhiều hệ điều hành hiện đại để cân bằng interactive và CPU-bound processes.

Liên hệ giữa các thuật toán lập lịch CPU
RR → FCFS: Nếu time quantum q rất lớn (lớn hơn mọi CPU burst), RR sẽ trở thành FCFS vì tiến trình chạy đến khi hoàn thành mới nhường CPU.
SRTF → SJF: SRTF là phiên bản preemptive của SJF; nếu không cho phép preemption thì SRTF trở thành SJF.
SJF → Priority Scheduling: Có thể xem SJF là một trường hợp đặc biệt của Priority Scheduling, trong đó priority được xác định bằng CPU burst nhỏ hơn.
MLFQ → RR + Priority + Aging: MLFQ kết hợp nhiều ý tưởng: dùng RR trong từng queue, ưu tiên giữa các queue và aging để tránh starvation.
MLQ vs MLFQ: MLQ có queue cố định, tiến trình không đổi queue; MLFQ cho phép di chuyển giữa các queue nên linh hoạt hơn.
Preemptive vs Non-preemptive: FCFS, SJF thường là non-preemptive; RR, SRTF và đa số Priority Scheduling hiện đại là preemptive.

Chapter 6: Synchronization Tools

Race Condition: Xảy ra khi nhiều tiến trình/luồng truy cập dữ liệu chung đồng thời làm kết quả phụ thuộc thứ tự thực thi

```
do {  
    entry section      // Xin vào CS  
    critical section    // Vùng dùng chung  
    exit section        // Rời CS  
    remainder section  // Phần còn lại  
} while(true);
```

Ba điều kiện giải quyết Critical Section
Mutual Exclusion: chỉ một tiến trình trong CS tại một thời điểm
Progress: tiến trình muốn vào CS không bị trì hoãn vô hạn
Bounded Waiting: giới hạn số lần tiến trình khác được vào trước mình

Giải pháp
Peterson Solution: Giải pháp phần mềm cho 2 tiến trình dùng flag và turn, Đáp ứng Mutual Exclusion, Progress, Bounded Waiting, Chủ yếu dùng minh họa lý thuyết, Không phù hợp kiến trúc hiện đại do instruction reordering
Memory Barrier: Ngăn CPU/Compiler reorder instruction, Buộc thay đổi bộ nhớ visible cho processor khác, Quan trọng trong multicore system
Test-and-Set: Lệnh phần cứng nguyên tử kiểm tra và cập nhật lock, Thường dùng xây mutex hoặc spinlock, Đảm bảo thao tác không bị gián đoạn
Compare-and-Swap (CAS): Chỉ cập nhật khi giá trị đúng trạng thái mong muốn, Nền tảng của lock-free structure, Giảm nhu cầu dùng lock truyền thống
Atomic Variable: Hỗ trợ thao tác nguyên tử trên dữ liệu cơ bản, Đảm bảo thread-safe cho increment/counter, Thường dùng cho state hoặc flag
Mutex Lock: Lock nhị phân dùng acquire/release bảo vệ CS, Acquire và Release phải nguyên tử, Busy waiting tạo thành spinlock
Semaphore: Biến đếm hỗ trợ synchronization bằng wait/signal, Hỗ trợ block/wakeup tránh busy waiting, Dùng cho synchronization và mutual exclusion
Binary Semaphore: Giá trị chỉ từ 0 hoặc 1, Hoạt động gần giống mutex, Thường dùng cho mutual exclusion
Counting Semaphore: Cho phép nhiều giá trị nguyên, Quản lý nhiều tài nguyên cùng lúc, Phù hợp resource pool hoặc producer-consumer
Monitor: Cơ chế đồng bộ mức cao chỉ cho một tiến trình hoạt động bên trong, Ẩn lock khỏi programmer, Ít lỗi hơn semaphore
Condition Variable: Cho phép wait() chờ điều kiện, Signal() dùng đánh thức tiến trình chờ, Thường đi kèm mutex hoặc monitor